

GPU-Accelerated Matrix Multiplication and Image Convolution in CUDA

DSCI 560 - Lab 3

Github Link: https://github.com/ShamikOfficial/Lab3_CUDA

Set up Environment

Before starting this lab, we set up a connection between VisualStudioCode and Google Colab using the Colab extension. This allowed us to work in VS Code while running code on Colab's remote environment, giving us access to an NVIDIA Tesla T4 GPU for the lab. Or alternatively, you can use google colab and connect to the Tesla T4 GPU there, and it would possibly make it easier for some who have a hard time uploading images through Visual Studio for Part 2.

```
# check if GPU is available
!nvidia-smi
```

✓ 0.1s

Fri Jan 30 23:30:31 2026

NVIDIA-SMI 550.54.15		Driver Version: 550.54.15		CUDA Version: 12.4	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util Compute M.
					MIG M.
0	Tesla T4	Off	00000000:00:04.0	Off	0
N/A	34C	P8	9W / 70W	0MiB / 15360MiB	0% Default
					N/A

Processes:

GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
	ID	ID				
No running processes found						

```
# check nvcc version
!nvcc --version
```

✓ 0.1s

nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Jun__6_02:18:23_PDT_2024
Cuda compilation tools, release 12.5, V12.5.82
Build cuda_12.5.r12.5/compiler.34385749_0

Part 1: Matrix Multiplication on the CPU

(1) Matrix Multiplication on the CPU

In this part, we implemented matrix multiplication using `matrix_cpu.c` and measured the CPU execution time. Since the example code is provided, we focused on understanding the C syntax, CUDA, and the structure of the example code.

The example defined a `matrixMultiplyCPU()` function to calculate the $N \times N$ matrix multiplication and measure the CPU's running time. It allocates the main memory for 3 $N \times N$ float arrays A and B. A and B are initialized with random values. And then it calls `clock()` to record the start and end times of the multiplication. After finishing the timing, the program released the memory of the CPU. We wrote the example code into the `matrix_cpu.c` file.

The `matrix_cpu.c` file was compiled with `gcc matrix_cpu.c -o matrix_cpu -O2`. In order to get more runtime measurements for N values, we defined a helper function named `run_time_comparasion()` to record the runtime for each CPU and GPU. It takes 2 parameters including a N value list and an executable file name. The N values we measured include 64, 128, 256, 512, 1024, and 2048.

(2) Naive CUDA Kernel

In this part, we implemented a CUDA kernel where each thread computes one element of the output matrix.

A CUDA kernel `matrixMultiplyGPU` is defined to compute matrix multiplication on the GPU. Each thread computes its global row and column indices using `blockIdx`, `blockDim`, and `threadIdx`, and then calculates one output element C by adding the dot product of row `row` in A and column `col` in B.

In the `main()` function, the matrix dimension N is read from the given N and the required memory size for $N \times N$ matrix is calculated as a size variable. And then the host memory pointers `h_A`, `h_B` and `h_C` are defined, where `h_A` and `h_B` are initialized with random values and `h_C` is set as 0s. The corresponding device memory pointers `d_A`, `d_B` and `d_C` are set as null and are allocated memory on GPU using `cudaMalloc`. After that, the `h_A` and `h_B` are copied from host to device using `cudaMemcpy`. The block size is set as 16×16 and grid size is calculated by ceiling division from the matrix dimension. A warm-up kernel launch is executed to reduce the overhead of the first launch, followed by error checking and synchronization. Kernel execution time is then measured using CUDA events by recording timestamps of starting and ending points of the kernel execution. Finally, the events are destroyed and both device and host memory are freed.

Issues Faced: When compiling the .cu file with `!nvcc matrix_naive_gpu.cu -o matrix_naive_gpu`, we encountered an error: Kernel launch error: the provided PTX was compiled with an unsupported toolchain.

The error is because the PTX generated by nvcc is not compatible with the GPU driver on the execution machine, so the kernel cannot be loaded or JIT-compiled. Adding `-arch=sm_75` compiles the code for the target GPU architecture, ensuring matching device code is generated and reducing PTX compatibility issues, which makes the kernel launch succeed.

Therefore, we using the command below to compile the CUDA file:

```
!nvcc -O2 -arch=sm_75 matrix_naive_gpu.cu -o matrix_naive_gpu
```

All the CUDA files followed are compiled with the `-arch=sm_75` parameter.

(3) Optimizing CUDA Code

In this part, we introduced tiling with shared memory to reduce global memory accesses. Shared memory is faster to access than global memory because it is on-chip storage located within each Streaming Multiprocessor (SM). Since global memory is stored on device DRAM, shared memory offers much lower latency and it can be accessed by all threads inside the block. In naive CUDA, many threads repeatedly load the same A and B elements from global memory to execute matrix multiplication, which causes repeated access to global memory.

With a tile width of 16, each block loads a 16×16 tile of A and B into shared memory once, and all threads in the block can load these cached values on the chip. This reduces redundant global memory traffic and improves overall throughput, leading to shorter GPU execution time.

Memory Model

Registers & Local Memory

- Regular variables declared within a Kernel

Shared Memory

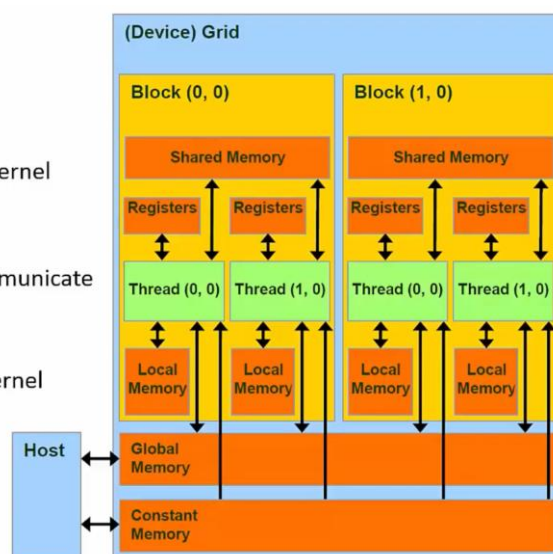
- Allows threads within a block to communicate

Constant

- Used for unchanging data through Kernel

Global Memory

- Stores data copied to and from Host



<https://youtu.be/OSpy-HoR0ac?si=Fbw6YUmEHfEm0DZm>

(4) Performance Comparison

We saved all the runtimes of CPU and GPU into a DataFrame and calculated the speedup = CPU time / GPU time.

For this stage the performance tables are listed below. More detailed explanation will be discussed after the cuBLAS implementation.

Implementation	N=64	N=128	N=256	N=512	N=1024	N=2048
CPU (C)	0.0004 sec	0.0032 sec	0.0212 sec	0.2060 sec	3.4606 sec	80.1934 sec
Naive CUDA	0.0155 ms	0.0324 ms	0.1867 ms	1.1592 ms	9.1939 ms	67.0700 ms
Optimized CUDA	0.0102 ms	0.0369 ms	0.0696 ms	0.7476 ms	5.8040 ms	46.2909 ms

	N	Speedup (CPU / Naive CUDA)	Speedup (CPU / Optimized CUDA)
0	64	25.806452	39.215686
1	128	98.765432	86.720867
2	256	113.551152	304.597701
3	512	177.708765	275.548422
4	1024	376.401745	596.243970
5	2048	1195.667213	1732.379366

(5) Using cuBLAS Library

CuBLAS is NVIDIA's GPU-accelerated implementation of the standard Basic Linear Algebra Subprograms (BLAS) library. It provides optimized routines for linear algebra operations on NVIDIA GPUs using CUDA, such as the single-precision general matrix multiply(cublasSgemm).

In this part, we use cublasSgemm to calculate matrix multiplication instead of hand written CUDA kernel. In the main() function, a cuBLAS handle is created with cublasCreate. Constant variables alpha and beta are defined for the GEMM operation. A warm-up cublasSgemm call is executed to avoid first-call overhead, followed by cudaDeviceSynchronize(). The cuBLAS execution time is measured using CUDA events by recording start and stop around the cublasSgemm call and computing the elapsed time in milliseconds.

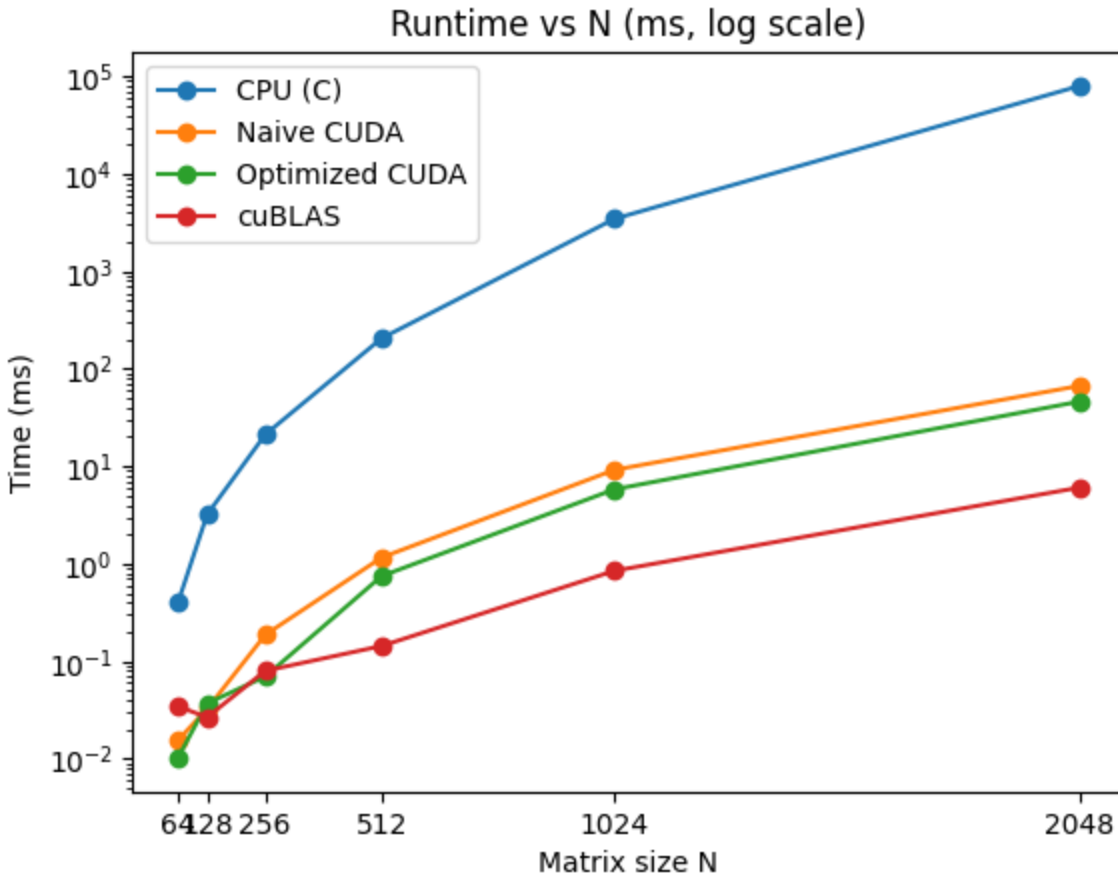
For compiling, we used this command: `lnvcc -O2 -arch=sm_75 matrix_cublas_gpu.cu -o matrix_cublas_gpu -lcublas`

(6) Overall Performance Comparison

In this part, we saved all the CPU and GPU's runtimes into a Dataframe. After converting the seconds into milliseconds, we calculated all the speedup. Besides, we plotted the execution time vs. matrix size for all CPU and GPU implementations.

Implementation	N=64	N=128	N=256	N=512	N=1024	N=2048
CPU (C)	0.0004 sec	0.0032 sec	0.0212 sec	0.2060 sec	3.4606 sec	80.1934 sec
Naive CUDA	0.0155 ms	0.0324 ms	0.1867 ms	1.1592 ms	9.1939 ms	67.0700 ms
Optimized CUDA	0.0102 ms	0.0369 ms	0.0696 ms	0.7476 ms	5.8040 ms	46.2909 ms
cuBLAS	0.0344 ms	0.0266 ms	0.0797 ms	0.1434 ms	0.8465 ms	6.0680 ms

	N	Speedup (CPU / Naive CUDA)	Speedup (CPU / Optimized CUDA)	Speedup (CPU / cuBLAS)
0	64	25.806452	39.215686	11.627907
1	128	98.765432	86.720867	120.300752
2	256	113.551152	304.597701	265.997491
3	512	177.708765	275.548422	1436.541144
4	1024	376.401745	596.243970	4088.127584
5	2048	1195.667213	1732.379366	13215.787739



Analysis Questions:

1. How does performance change as matrix size increases?

As expected, all implementations slow down as matrices get bigger, but they handle it very differently. The CPU struggles the most, with execution times slowing down more than the expected theoretical quadratic rate with large matrices possibly affected by the cache management. The GPU implementations surprisingly scales more smoothly, with cuBLAS showing the best result on scaling. cuBLAS somehow manages to perform faster with growing matrix size instead of slowing down.

2. At what point does the GPU significantly outperform the CPU?

The GPU starts beating the CPU right from the smallest to the largest sizes we tested, and the advantage just keeps growing as matrices get bigger. By medium sizes, the GPU is already orders of magnitude faster, and that gap widens almost exponentially for large matrices where the CPU really starts to struggle.

3. How much speedup is gained by tiling optimization vs. naïve CUDA?

Tiling does not show much difference with very small sizes, as the overhead actually makes it slower. But, once you hit a certain size, the shared memory optimization kicks in and we see a noticeable upward trend

improvement. Tiling works well in a specific range of sizes and does not have any significant difference outside of it.

4. How close is your optimized kernel to cuBLAS performance?

For small matrices, our optimized kernel actually does pretty well against cuBLAS, sometimes even slightly better. But as matrices grow larger, cuBLAS gets significantly faster. The gap gets pretty significant for problems with large computations.

5. Why might cuBLAS still outperform hand-written kernels?

cuBLAS has years of optimization work behind it, with engineers who understand every detail of GPU architecture. It uses specialized hardware like tensor cores, adapts algorithms to different GPU generations, and applies low-level optimizations that are hard to replicate without deep domain expertise.

(7) Creating a Shared Library and Using it in Python

The code wraps the optimized CUDA matrix multiplication kernel into a shared library (libmatrix.so) callable from Python. The extern "C" wrapper function `gpu_matrix_multiply()` handles GPU memory allocation, data transfers, and kernel execution. Python uses ctypes to load the library, declare argument types (NumPy float32 arrays and an integer), and call the function with flattened NumPy arrays, enabling GPU acceleration from Python without rewriting CUDA code.

Key Components:

CUDA Library (matrix_lib.cu): Contains the tiled matrix multiplication kernel and an extern "C" wrapper that manages GPU memory and execution.

Compilation: Uses `nvcc -Xcompiler -fPIC -shared` to create a position-independent shared library (.so file).

Python Interface: Uses ctypes to load the library, specify argument types matching NumPy arrays, and call the CUDA function directly from Python, achieving GPU acceleration with minimal overhead.

Part 2: Adding Custom Functions to the Shared Library

Step 1: Convolution Function

Convolution filter (kernel) is a N by N matrix that is used for image processing. The image processing is done by a weighted sum of a pixel value and its neighborhood NxN pixel values to create a new pixel value. This feature makes this a good candidate for using CUDA since GPU is great for performing weighted sum with big data.

I will be using the following kernel filters:

- **Edge Detection**
 - Edge detection was asked from us specifically.
- **Gaussian Blur**
 - I chose Gaussian blur instead of the normal blur since normal blur was not very obvious to the eye.
- **Sharpen**

We selected the kernel weights from the Wikipedia article on image-processing kernels:

[https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)).

Image Preprocessing

I have used the Python Pillow library to preprocess the image to 1024 to 1024. Then I've used the `convert("L")` method in the Pillow (PIL) library to convert an image to 8-bit grayscale mode so that the image is monochrome.

Code for preprocessing:

```
from PIL import Image
img = Image.open("image.jpg").convert("L").resize((1024, 1024))
img.save("input.pgm")
```

First Image before resizing and grayscaling:



Resized (1024, 1024) Monochrome Image:



The Code to Perform Convolution Function:

We have implemented convolution using nested loops where we iterate every pixel in the vector and for each pixel we get the neighborhood pixels to create the NxN square to perform the filter. Each filter function has different weights of course and we basically define NxN, and then do weighted sum and store the result. The helper function `get_pixel_zero_ped()` understands the boundaries and returns zero when the neighboring pixel is out of bounds. We define the matrix bounds by the defined weight (by default M=512, but it can be changed with inputting arguments in the CLI) length and at each weight length, it switches to the new row in the matrix.

Example Code for N=3 edge Detection:

```
// EDGE N=3:
static void edge_n3(const uint8_t *image, uint8_t *out, int W, int H) {
    const int32_t K[9] = {
        0, -1, 0,
        -1, 4, -1,
        0, -1, 0
    };
    int r = 1;

    for (int y = 0; y < H; y++) {
        for (int x = 0; x < W; x++) {
            int32_t sum = 0;

            for (int ky = 0; ky < 3; ky++) {
                for (int kx = 0; kx < 3; kx++) {
                    int ix = x + (kx - r);
                    int iy = y + (ky - r);
                    uint8_t p = get_pixel_zero_pad(image, W, H, ix, iy);
                    sum += (int32_t)p * K[ky * 3 + kx];
                }
            }

            if (sum < 0) sum = -sum;
            out[y * W + x] = clamp_to_u8(sum);
        }
    }
}
```

Filters:

As mentioned above, we have 3 different filters which are edge detection, sharpen, and gaussian blur. We have used the filter weights directly from the wikipedia website for 3x3 ones. For 5x5 and 7x7 ones, we extended the same patterns used in the 3x3 one. We used the laplacian style (big center, rest is negative ones) for the edge detection. For Sharpen and Gaussian blur weights, we checked the standard solutions from academia resources. We also normalized the gaussian blur to keep brightness normal.

Which filter to run is defined by mode string value such as “edge_n3”, or “sharpen_n7”, so we can test different kernels by just changing the argument in CLI.

Running the Code:

```
! ./convo input.pgm out_edge_n5_256.pgm    edge_n5    256
! ./convo input.pgm out_edge_n7_256.pgm    edge_n7    256

! ./convo input.pgm out_sharp_n3_256.pgm    sharpen_n3 256
! ./convo input.pgm out_sharp_n5_256.pgm    sharpen_n5 256
```

This will generate two things: one it will print out the time it took to perform the convolution with all necessary information such as the N, M, and the filter used (mode). Then it will generate the filtered image with the name you defined “out_edge_nx_M.pgm” from the inputted image of “input.pgm”.

Performance of CPU only:

```
Compute time (convolution only): mode=edge_n3, N=3, M=256 -> 1.202 ms
Compute time (convolution only): mode=edge_n5, N=5, M=256 -> 2.388 ms
Compute time (convolution only): mode=edge_n7, N=7, M=256 -> 7.079 ms
Compute time (convolution only): mode=sharpen_n3, N=3, M=256 -> 1.394 ms
Compute time (convolution only): mode=sharpen_n5, N=5, M=256 -> 2.861 ms
Compute time (convolution only): mode=sharpen_n7, N=7, M=256 -> 4.659 ms
Compute time (convolution only): mode=blur_n7, N=7, M=256 -> 7.633 ms
Compute time (convolution only): mode=blur_n3, N=3, M=256 -> 1.347 ms
Compute time (convolution only): mode=blur_n5, N=5, M=256 -> 2.562 ms
Compute time (convolution only): mode=edge_n3, N=3, M=512 -> 5.388 ms
Compute time (convolution only): mode=edge_n5, N=5, M=512 -> 9.901 ms
Compute time (convolution only): mode=edge_n7, N=7, M=512 -> 27.389 ms
Compute time (convolution only): mode=sharpen_n3, N=3, M=512 -> 4.492 ms
Compute time (convolution only): mode=sharpen_n5, N=5, M=512 -> 10.415 ms
Compute time (convolution only): mode=sharpen_n7, N=7, M=512 -> 18.694 ms
Compute time (convolution only): mode=blur_n3, N=3, M=512 -> 7.518 ms
Compute time (convolution only): mode=blur_n5, N=5, M=512 -> 10.343 ms
Compute time (convolution only): mode=blur_n7, N=7, M=512 -> 27.197 ms
Compute time (convolution only): mode=edge_n3, N=3, M=1024 -> 20.132 ms
Compute time (convolution only): mode=edge_n5, N=5, M=1024 -> 39.494 ms
Compute time (convolution only): mode=edge_n7, N=7, M=1024 -> 111.012 ms
Compute time (convolution only): mode=sharpen_n3, N=3, M=1024 -> 16.895 ms
Compute time (convolution only): mode=sharpen_n5, N=5, M=1024 -> 40.281 ms
Compute time (convolution only): mode=sharpen_n7, N=7, M=1024 -> 76.936 ms
Compute time (convolution only): mode=blur_n3, N=3, M=1024 -> 22.251 ms
Compute time (convolution only): mode=blur_n5, N=5, M=1024 -> 55.309 ms
Compute time (convolution only): mode=blur_n7, N=7, M=1024 -> 195.043 ms
```

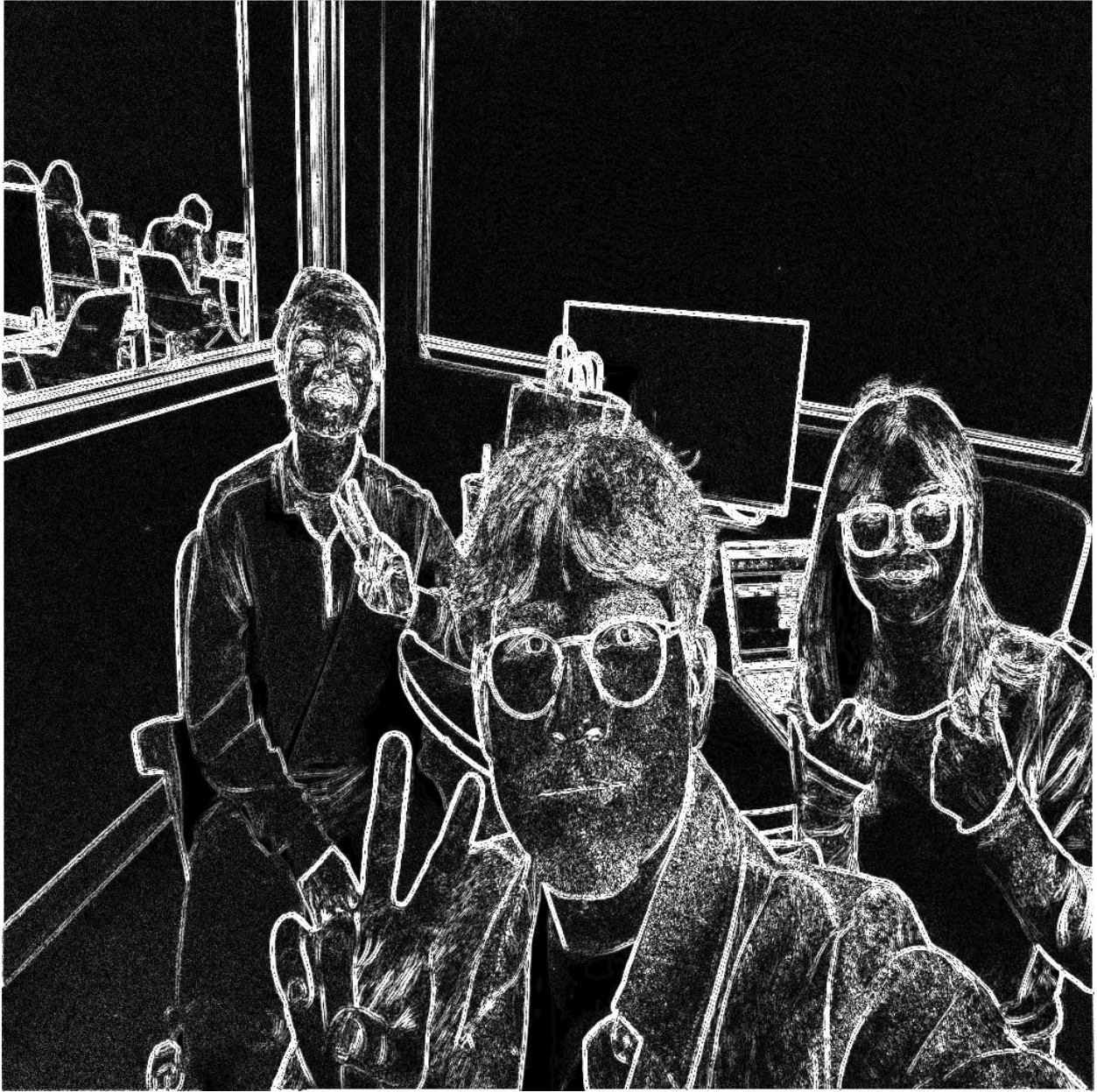
The time increases as M increases. The time also increases as N increases. The time is not equal for all filters.

Displaying Images to prove Functions work Properly:

- Edge n3 with 1024 size:



- Edge n5 with 1024 size:



- Sharp n5 1024 size:



- Blur n7 with 512 size:



As you can see, the image is cropped to the top-left corner to be able to fit to the smaller size of 512 we have defined.

The results from other 2 images are inside the GitHub repository.

Step 2: Porting the Convolution Function to CUDA

Why this is a good CUDA Candidate:

This is highly parallel because output pixels can be computed independently so we are basically applying the same math but different pixels at each time. As mentioned in the class, GPU's are designed exactly for this kind of repeated multiplication and summation (in parallel) across large arrays so CUDA should help here.

Essentially the same kernel functions. GPU processing allocates device memory with using `cudaMalloc` and each CUDA kernel uses a grid of threads, where one thread computes one output pixel (x, y) by

looping over the NxN neighborhoods we have, multiplying by the kernel weights we have assigned in filter functions, applying boundary checks, and clipping the final value to the 0-255.

GPU Performance:

```
...  CUDA kernel time (events): mode=edge_n3, N=3, M=256 -> 0.208 ms
      CUDA kernel time (events): mode=edge_n5, N=5, M=256 -> 0.213 ms
      CUDA kernel time (events): mode=edge_n7, N=7, M=256 -> 0.255 ms
      CUDA kernel time (events): mode=sharpen_n3, N=3, M=256 -> 0.206 ms
      CUDA kernel time (events): mode=sharpen_n5, N=5, M=256 -> 0.216 ms
      CUDA kernel time (events): mode=sharpen_n7, N=7, M=256 -> 0.237 ms
      CUDA kernel time (events): mode=blur_n7, N=7, M=256 -> 0.238 ms
      CUDA kernel time (events): mode=blur_n3, N=3, M=256 -> 0.217 ms
      CUDA kernel time (events): mode=blur_n5, N=5, M=256 -> 0.225 ms
      CUDA kernel time (events): mode=edge_n3, N=3, M=512 -> 0.208 ms
      CUDA kernel time (events): mode=edge_n5, N=5, M=512 -> 0.236 ms
      CUDA kernel time (events): mode=edge_n7, N=7, M=512 -> 0.279 ms
      CUDA kernel time (events): mode=sharpen_n3, N=3, M=512 -> 0.223 ms
      CUDA kernel time (events): mode=sharpen_n5, N=5, M=512 -> 0.232 ms
      CUDA kernel time (events): mode=sharpen_n7, N=7, M=512 -> 0.278 ms
      CUDA kernel time (events): mode=blur_n3, N=3, M=512 -> 0.218 ms
      CUDA kernel time (events): mode=blur_n5, N=5, M=512 -> 0.256 ms
      CUDA kernel time (events): mode=blur_n7, N=7, M=512 -> 0.281 ms
      CUDA kernel time (events): mode=edge_n3, N=3, M=1024 -> 0.199 ms
      CUDA kernel time (events): mode=edge_n5, N=5, M=1024 -> 0.293 ms
      CUDA kernel time (events): mode=edge_n7, N=7, M=1024 -> 0.604 ms
      CUDA kernel time (events): mode=sharpen_n3, N=3, M=1024 -> 0.245 ms
      CUDA kernel time (events): mode=sharpen_n5, N=5, M=1024 -> 0.294 ms
      CUDA kernel time (events): mode=sharpen_n7, N=7, M=1024 -> 0.441 ms
      CUDA kernel time (events): mode=blur_n3, N=3, M=1024 -> 0.220 ms
      CUDA kernel time (events): mode=blur_n5, N=5, M=1024 -> 0.310 ms
      CUDA kernel time (events): mode=blur_n7, N=7, M=1024 -> 0.453 ms
```

Performance: Comparison of CPU and GPU

Collected 9 CPU modes and 9 CUDA modes

	M=256		M=512		M=1024	
Filter						
edge_n3	CPU: 1.198 ms, CUDA: 0.217 ms (Speedup: 5.52x)	CPU: 5.216 ms, CUDA: 0.253 ms (Speedup: 20.62x)	CPU: 20.372 ms, CUDA: 0.189 ms (Speedup: 107.79x)			
edge_n5	CPU: 2.509 ms, CUDA: 0.268 ms (Speedup: 9.36x)	CPU: 10.228 ms, CUDA: 0.275 ms (Speedup: 37.19x)	CPU: 39.561 ms, CUDA: 0.308 ms (Speedup: 128.44x)			
edge_n7	CPU: 7.384 ms, CUDA: 0.234 ms (Speedup: 31.56x)	CPU: 28.293 ms, CUDA: 0.284 ms (Speedup: 99.62x)	CPU: 113.929 ms, CUDA: 0.442 ms (Speedup: 257....)			
sharpen_n3	CPU: 1.063 ms, CUDA: 0.221 ms (Speedup: 4.81x)	CPU: 4.169 ms, CUDA: 0.248 ms (Speedup: 16.81x)	CPU: 17.123 ms, CUDA: 0.259 ms (Speedup: 66.11x)			
sharpen_n5	CPU: 2.626 ms, CUDA: 0.242 ms (Speedup: 10.85x)	CPU: 12.244 ms, CUDA: 0.266 ms (Speedup: 46.03x)	CPU: 45.814 ms, CUDA: 0.392 ms (Speedup: 116.87x)			
sharpen_n7	CPU: 5.140 ms, CUDA: 0.254 ms (Speedup: 20.24x)	CPU: 19.389 ms, CUDA: 0.302 ms (Speedup: 64.20x)	CPU: 81.197 ms, CUDA: 0.451 ms (Speedup: 180.04x)			
blur_n3	CPU: 1.452 ms, CUDA: 0.297 ms (Speedup: 4.89x)	CPU: 5.618 ms, CUDA: 0.286 ms (Speedup: 19.64x)	CPU: 22.442 ms, CUDA: 0.270 ms (Speedup: 83.12x)			
blur_n5	CPU: 2.580 ms, CUDA: 0.354 ms (Speedup: 7.29x)	CPU: 10.989 ms, CUDA: 0.323 ms (Speedup: 34.02x)	CPU: 43.981 ms, CUDA: 0.382 ms (Speedup: 115.13x)			
blur_n7	CPU: 7.280 ms, CUDA: 0.376 ms (Speedup: 19.36x)	CPU: 29.059 ms, CUDA: 0.358 ms (Speedup: 81.17x)	CPU: 115.875 ms, CUDA: 0.522 ms (Speedup: 221....)			

--- Summary Statistics ---
Average speedup: 67.05x
Minimum speedup: 4.81x
Maximum speedup: 257.76x

Step 3: Adding and Using the Convolution in the Custom Python Library

Compiles the CUDA convolution kernels into a shared library (libconvo.so) that Python can call via ctypes. This lets Python scripts use GPU-accelerated convolution (edge detection, sharpening, blur) without rewriting the CUDA code, enabling performance comparisons between CPU, CUDA executables, and Python-bound CUDA functions.

GPU Performance per function in different architecture:

	Mode	M	C (ms)	CUDA Exec (ms)	Python+CUDA (ms)	Speedup (C vs CUDA Exec)	Speedup (C vs Python+CUDA)
0	edge_n3	256	1.217	0.206	76.478	5.91x	0.02x
1	sharpen_n3	256	1.027	0.219	0.691	4.69x	1.49x
2	blur_n3	256	1.418	0.206	0.603	6.88x	2.35x
3	edge_n3	512	4.888	0.214	0.953	22.84x	5.13x
4	sharpen_n3	512	4.004	0.235	0.851	17.04x	4.71x
5	blur_n3	512	5.419	0.233	0.829	23.26x	6.53x
6	edge_n3	1024	22.758	0.197	2.249	115.52x	10.12x
7	sharpen_n3	1024	16.311	0.200	1.869	81.55x	8.73x
8	blur_n3	1024	21.644	0.202	1.536	107.15x	14.09x

=== Summary ===

Performance comparison: C program vs CUDA executable vs Python+CUDA library

Note: Python+CUDA includes overhead of Python/ctypes interface